LAKESIDE MEDICAL · 4 CLASSES

Build a real clinic booking app in VS Code

Node.js + Express · Copilot Chat or Claude Code · SQLite

Jay · October 2026

THE BUILD

1. Project + landing page
2. Booking engine + database
3. Payments + email
4. AI, reception + publish

COURSE MAP

Four classes, one working clinic app



CLASS 1

Project + landing page

- Node + Express project
- Hero, specialties, contact
- First GitHub commit



CLASS 2

Booking engine

- 4-step booking flow
- SQLite database
- Patient login



CLASS 3

Payments + email

- Stripe test-mode deposit
- Confirmation email
- One-click reminder



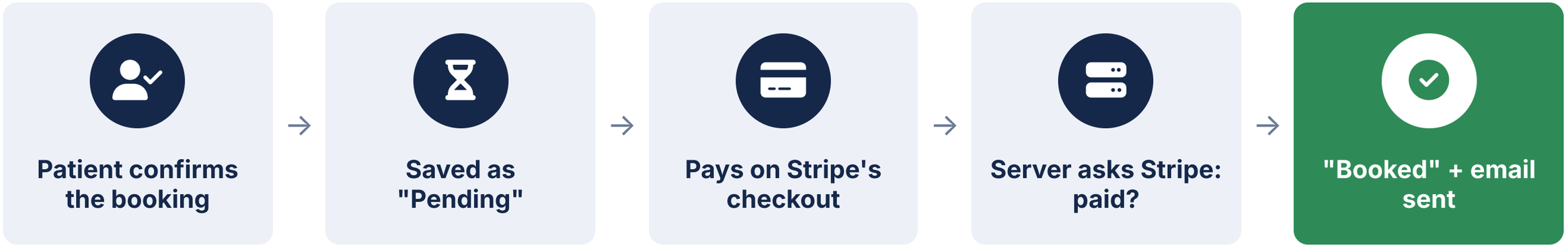
CLASS 4

AI, reception + publish

- Grounded AI assistant
- Staff-only dashboard
- Live on Azure

THE FLOW AT A GLANCE

A booking is only Booked once Stripe says paid



Think of it like this

A restaurant only confirms your table once the deposit actually clears, not because you said you paid. Your server checks with Stripe; it never trusts the browser redirect.

Not paid → stays "Pending"

Class 4 wraps this flow with the AI assistant on every page and the password-protected reception dashboard.

No new .exe, nothing for Windows to block

The .NET problem

Every .NET build creates a brand-new, unsigned `LakesideMedical.exe`. Smart App Control can block it, the server never starts, and the browser shows `ERR_CONNECTION_REFUSED`.

The Node fix

Your app is plain JavaScript files run by `node.exe`, which is already signed and trusted. No new program is created, so nothing is blocked.

Pure-JavaScript packages only

We use Node's built-in SQLite and `bcryptjs`, so no package compiles a native file that could be blocked too.

THINK OF IT LIKE THIS

.NET hands Windows a new key every time you build. Node uses the one key Windows already trusts, and just changes the instructions it reads.

```
.NET   build → new .exe   → blocked
Node   node.exe + .js     → runs
```

BEFORE CLASS 1 · ABOUT 20 MINUTES

Set up once, use in every class

Node.js LTS + VS Code

Install Node.js 22 LTS or newer from nodejs.org, and Visual Studio Code. No database to install: SQLite is built into Node.

GitHub + an AI coding agent

Sign in to GitHub in VS Code. Use Copilot Chat in Agent mode, or the Claude Code extension, so the agent edits the files itself.

Class 3: Stripe + email

A free Stripe account in test mode, plus an SMTP account: Gmail with an app password, or a Mailtrap sandbox inbox.

Class 4: AI API key

An API key from an AI provider, for example the Claude API.

HOW TO READ THE SCRIPT

Fill the brackets, keep everything else

[Square brackets]

Yours to fill in: clinic, area, doctors, your name. Replace the whole bracket, example included.

Plain text

Keep it exactly as written, so every student's project lines up.

{Curly braces}

Filled in automatically from the booking. Keep them too.

One prompt at a time

Paste, let the agent finish, then open `localhost:3000` and click through. Error in the terminal? Paste it back into the chat.

WHAT IT LOOKS LIKE

```
Clinic: [clinic name — e.g. Lakeside
Medical Clinic] in [area — e.g. JLT,
Dubai]. Use views/index.ejs as the single
landing page…

Body: "Hi {PatientName}, a reminder that
your {Specialty} appointment with {Doctor}
is on {Date} at {Time}…"
```

What replaces what

In the Lovable script	In this VS Code + Node script
Lovable project	A folder in VS Code (Node.js + Express)
Pages	EJS views in views/, routes in routes/
localStorage, then Supabase	SQLite (built into Node), then a file on Azure
storage.ts	services/appointmentService.js
Demo-only local login	express-session + bcryptjs (hashed passwords)
Project secrets	.env file, kept out of GitHub by .gitignore
Edge function	Server-side Express route
Lovable Emails	SMTP through the Nodemailer package
Connect GitHub	Source Control → Publish to GitHub
Publish in Lovable	Azure App Service extension → Deploy

CLASS 1

Create the project and the landing page



TODAY'S PROMPTS

- **P1** Landing page
- **P2** Logo + sticky navbar
- **P3** Polish the layout

Create the project

1



Make a folder

Create an empty folder called `lakeside-medical`.

2



Open it in VS Code

File → Open Folder → pick `lakeside-medical`. Trust the folder.

3



Check Node

Terminal → New Terminal. Type `node -v`: it should show v22 or higher.

4



Open the agent

Copilot Chat in Agent mode, or Claude Code. Paste the prompts one at a time.



No login yet. Patient accounts arrive in Class 2 with hashed passwords.

Three prompts build the public website

P1 · Create the landing page

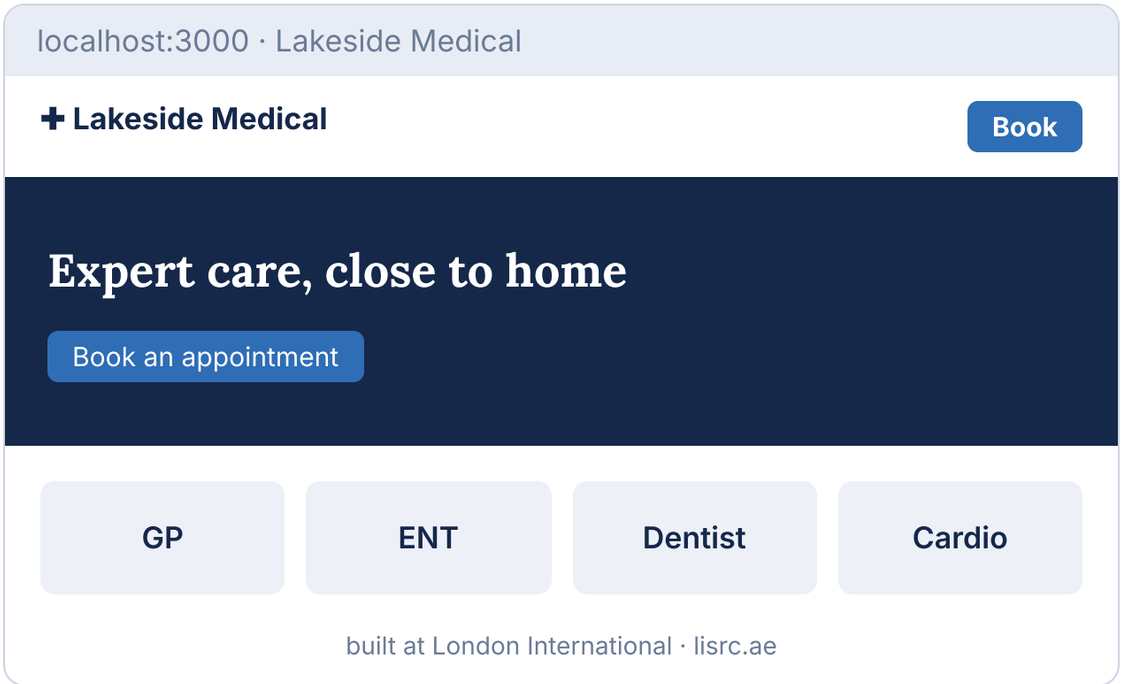
Sets up the Node project, then hero, four specialty cards, About, Contact and footer in one go.

P2 · Logo + sticky navbar

Logo and menu stay at the top while scrolling. "Book an appointment" starts the Class 2 flow.

P3 · Polish the layout

Equal-height cards, a gentle hover lift, and a full-width button on phones.



CLASS 1 · CHECKPOINT

Run it, then make your first commit

Open localhost:3000

Check it in a narrow window

Source Control → Publish to GitHub

Name it lakeside-medical

Public or private → Publish

Words you'll see in the prompts

Express
The small web server that answers each page request

EJS view
An HTML page with gaps the server fills in

Partial
header.ejs / footer.ejs: navbar + footer on every page

Hero
Big top banner with name, tagline, button

npm run dev
Starts the site and restarts it when you save

Commit
A saved snapshot of your work on GitHub

P1 · Create the landing page

Create a new Node.js website in this empty folder using Express and EJS templates. Clinic: **[clinic name — e.g. Lakeside Medical Clinic]** in **[area — e.g. JLT, Dubai]**. Set up package.json with the scripts "dev": "node --watch server.js" and "start": "node server.js", and install only express and ejs. Create server.js on port 3000 (or process.env.PORT), views/index.ejs as the single landing page, views/partials/header.ejs and views/partials/footer.ejs shared by every page, and public/css/site.css for custom styles. The landing page has: a hero (clinic name, tagline **[tagline — e.g. Expert care, close to home]**, a "Book an appointment" button); an "Our Specialties" section with four cards — General Physician, ENT, Dentist, Cardiologist (each a one-line description + a Bootstrap Icons icon); an "About Us" section; a "Contact" section (**[address — e.g. Cluster X, JLT, Dubai]**; phone; hours Mon–Sat 9am–9pm, closed Sunday); and a footer in footer.ejs. In the footer add a small line: **[clinic name]** · built at London International · lisrc.ae · by **[your name]**. Load Bootstrap 5 and Bootstrap Icons from a CDN. Clean medical theme in blue and white. Fully mobile-friendly. Add a .gitignore with node_modules, .env and data/*.db. Do not add a database yet. When done, run npm run dev and confirm http://localhost:3000 loads.

[Gold] = yours to fill in {Blue} = filled in automatically Everything else: keep as written

P2 · Add the logo and sticky navbar

> In views/partials/header.ejs, add a text logo **[clinic name — e.g. Lakeside Medical]** with a small medical icon in a sticky navbar: Home, Specialties, About, Contact (each scrolls to its section on the landing page), and the "Book an appointment" button on the right.

[Gold] = yours to fill in {Blue} = filled in automatically Everything else: keep as written

P3 · Polish the layout

> Make the four specialty cards equal height with a subtle hover lift; on mobile make the hero "Book an appointment" button full-width.

[Gold] = yours to fill in {Blue} = filled in automatically Everything else: keep as written

CLASS 2

The booking engine, saved in SQLite



TODAY'S PROMPTS

- **P1** Booking flow
- **P2** Save in the database
- **P3** Patient login

From screens to a real database to patient accounts

1. Specialty → 2. Doctor → 3. Date & time → 4. Your details → 5. Confirm



P1 · Booking flow

The patient-facing screens. Nothing is saved yet; only the doctor names are yours.



P2 · Save in the database

Each confirmed booking becomes one row in the appointments table, through appointmentService.



P3 · Login + My Appointments

Hashed-password accounts. Patients see only their own bookings; Cancel sets status to "Cancelled".

One appointment = one row in the database

```
appointments table

id
specialty
doctor
appointmentDateTime
patientName
patientPhone
patientEmail
status    ("Booked")
createdAt
```

Routes + EJS views
/booking, /my-appointments

↓

appointmentService.js
All database code in one file

↓

data/db.js
Opens the file, creates the tables

↓

SQLite (node:sqlite)
File: data/lakeside.db



A shared filing cabinet

Not a notebook in one browser: any page, and later any staff member, reads the same data. Routes never touch db.js directly, so the database can be swapped later.

See your booking as a row in the database

```
Extensions → search "SQLite Viewer" → Install
```
Lets you open the .db file inside VS Code

1. After P2, let the agent restart `npm run dev`; the tables are created automatically
2. Make a test booking
3. Click `data/lakeside.db` in the Explorer → appointments table
4. After P3: register, book, cancel, check the status column
5. Commit from the Source Control panel

Keep these exactly as written

`appointments` `appointmentService` `"Booked"` `"Cancelled"`

Change them and P2, P3 and Class 3 won't line up.

P1 · Build the booking flow

Add a booking flow at /booking (routes/booking.js + views/booking.ejs), opened by the "Book an appointment" buttons. Step 1: choose a specialty (General Physician, ENT, Dentist, Cardiologist). Step 2: choose a doctor — GP: **[doctor name — e.g. Dr. Sara Ahmed]**; ENT: **[doctor name — e.g. Dr. Omar Khan]**; Dentist: **[doctor name — e.g. Dr. Lina Haddad]**; Cardiologist: **[doctor name — e.g. Dr. Raj Mehta]**. Step 3: choose a date and a time slot (9am–9pm, 30-minute steps, no Sundays, no past dates). Step 4: enter patient name, phone, email, with validation on the server too. Show a summary and a "Confirm booking" button. Nothing is saved yet.

[Gold] = yours to fill in {Blue} = filled in automatically Everything else: keep as written

P2 · Save bookings in the database

Add a SQLite database using Node's built-in node:sqlite module. Do not install any database package and nothing that needs native compiling. Create data/db.js that opens the file at process.env.DB_PATH (default data/lakeside.db), creates the data folder if missing, and creates an appointments table if it does not exist with: id, specialty, doctor, appointmentDateTime, patientName, patientPhone, patientEmail, status (default "Booked"), createdAt. Put all database code in one file called services/appointmentService.js (createBooking, getBookingsByEmail, getBookingsByDate, updateStatus) using prepared statements, so the database can be swapped later. Routes must call appointmentService and never use db.js directly. On "Confirm booking", save the booking through appointmentService and show a confirmation screen with the details. Restart npm run dev and confirm a test booking is saved.

[Gold] = yours to fill in {Blue} = filled in automatically Everything else: keep as written

P3 · Patient login and My Appointments

Add patient sign-up and login with email + password using express-session and bcryptjs (pure JavaScript, no native modules). Create a users table in the same lakeside.db inside data/db.js, and store only hashed passwords. Create a .env file with SESSION_SECRET set to a long random value, and load it at the top of server.js with process.loadEnvFile() only if .env exists, so the app also runs on a server that uses environment variables. Add Register, Login and Logout links to the navbar. After login, add a "My Appointments" page at /my-appointments that shows only the logged-in patient's appointments, matched by patientEmail, newest first, each with a "Cancel" button that sets status to "Cancelled". A patient may only cancel their own appointments. Pre-fill the patient email on the booking form from the logged-in user.

[Gold] = yours to fill in {Blue} = filled in automatically Everything else: keep as written

CLASS 3

Payments and email



TODAY'S PROMPTS

- **P1** Stripe deposit
- **P2** Confirmation email
- **P3** Reminder email

Keys live in .env, never in a prompt

.env

```
SESSION_SECRET=a-long-random-value
BASE_URL=http://localhost:3000

STRIPE_SECRET_KEY=sk_test_...

SMTP_HOST=smtp.example.com
SMTP_PORT=587
SMTP_USER=your-smtp-username
SMTP_PASS=your-smtp-password
SMTP_FROM=clinic@example.com
```

Create a free Stripe account, switch to test mode, copy the key starting sk_test_

Open `.env` in the project root (P3 of Class 2 created it)

Add the lines with your own Stripe and SMTP values, save, restart `npm run dev`

.gitignore lists .env, so your keys are never committed to GitHub.

Deposit, confirmation, reminder

P1 · Stripe deposit

AED 100 refundable deposit (amount 10000). Saved "Pending", then "Booked" only if Stripe's payment_status is "paid".

Never trust the redirect alone

P2 · Confirmation email

Nodemailer over SMTP, all in emailService.js. Sent once per booking, even if the page is refreshed.

Subject: Your appointment is confirmed

P3 · Reminder email

A basic /reception page with a "Send reminder" button per booking. Cuts no-shows.

Keep {PatientName} {Doctor} {Date}...

Test it: pay once, then try to skip paying

TEST CARD



4242 4242 4242 4242

Any future date · any CVC · any postcode

Restart npm run dev after editing .env. Test mode moves no real money.

No SMTP account yet? Use a Mailtrap sandbox inbox: mail is caught there instead of delivered.



Pay with the test card

status becomes "Booked" in SQLite Viewer, and the confirmation lands in your inbox.



Click Stripe's back arrow instead

status must stay "Pending". If it says Booked, the server check is missing.



Click Send reminder

The reminder email arrives with your booking details filled in. Then commit.

Your checkout transfers: swap the provider



Test mode

Needs no licence. Everyone builds the full flow in class today.

To take real money

- **Stripe** with a trade licence
- **Mamo**
- **Ziina**
- **Telr**
- **PayTabs**
- **Network International**
- **Payment link** simplest start

Your trainer covers the go-live options in class.

P1 · Add the Stripe deposit

Add a Stripe deposit to the booking flow, in test mode, using the stripe npm package. Read the Stripe secret test key from process.env.STRIPE_SECRET_KEY in .env — never from this prompt, from code committed to GitHub, or from browser JavaScript. When the patient confirms the booking: first save the appointment with status "Pending", then, in a server-side Express route, create a Stripe Checkout Session for a one-off AED 100 refundable consultation deposit (currency aed, amount 10000, the smallest unit) and redirect the patient to the Stripe-hosted checkout page. Save the Checkout Session id on the appointment: add a stripeSessionId column to the appointments table in data/db.js, only if it is missing, so existing bookings are kept. Build the URLs from process.env.BASE_URL: success URL /booking-success?session_id={CHECKOUT_SESSION_ID} and cancel URL back to /booking. On the /booking-success route, find the appointment by stripeSessionId, retrieve the Checkout Session from Stripe on the server, and only set status to "Booked" if payment_status is "paid" — otherwise it stays "Pending". Never trust the redirect alone. Show the patient a clear confirmation, or a "payment not completed" message.

[Gold] = yours to fill in {Blue} = filled in automatically Everything else: keep as written

P2 · Send the confirmation email

When an appointment becomes "Booked" after a paid checkout, send a confirmation email using the nodemailer package over SMTP. Read SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS and SMTP_FROM from .env. Put all email code in one file called services/emailService.js. Send from "**[clinic name — e.g. Lakeside Medical]**" to the patientEmail saved with the booking (for testing, book with your own email so the confirmation lands in your inbox), with subject "Your appointment is confirmed" and a body with specialty, doctor, date, time, and the clinic address **[address — e.g. Cluster X, JLT, Dubai]**. End the email with a small sign-off line: "**[clinic name]** · built at London International · lisrc.ae · by **[your name]**". Send it only once per appointment, even if the success page is refreshed: add a confirmationEmailSentAt column to the appointments table in data/db.js, only if it is missing. If sending fails, log the error and still show the booking as confirmed.

[Gold] = yours to fill in {Blue} = filled in automatically Everything else: keep as written

P3 · Send the reminder email

Add a basic staff page at /reception that lists all appointments with status "Booked" (patient name, date and time, specialty, doctor), each with a "Send reminder" button. The button sends an email through emailService to the patientEmail saved with the booking (for testing, use a booking made with your own email), with subject "Reminder: your appointment" and body "Hi **{PatientName}**, a reminder that your **{Specialty}** appointment with **{Doctor}** is on **{Date}** at **{Time}** at **[clinic name + area — e.g. Lakeside Medical, JLT]**. Reply to reschedule." The words in curly braces are filled automatically from the booking. Add a small sign-off line at the end: "Powered by London International · lisrc.ae · built by **[your name]**".

[Gold] = yours to fill in {Blue} = filled in automatically Everything else: keep as written

CLASS 4

AI assistant, reception dashboard and publish



TODAY'S PROMPTS

- **P1** AI assistant
- **P2** Reception dashboard
- **P3** Mobile + publish

A grounded assistant with a hard guardrail

Grounded

Answers only from your clinic info: specialties, doctors, hours, insurance, location, deposit.

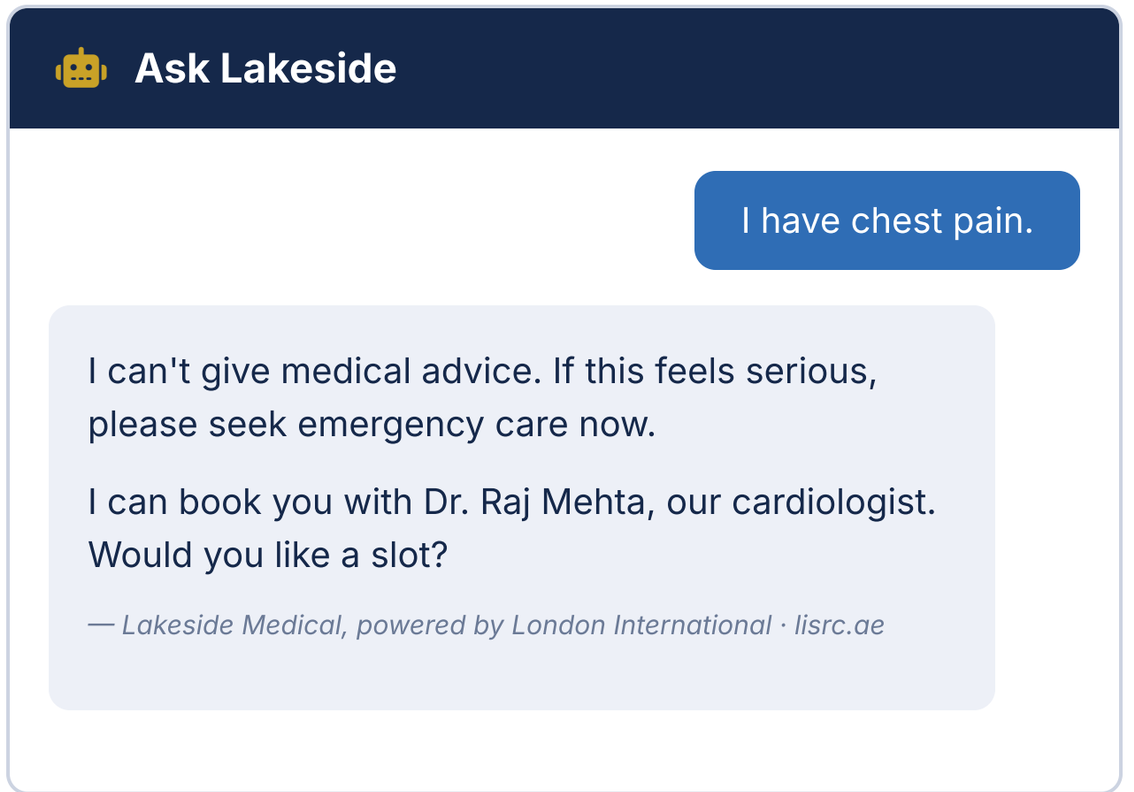
GUARDRAIL

No medical advice, symptoms, diagnoses or treatments. Offer to book the right specialist instead.

Key stays on the server

POST /api/chat reads ANTHROPIC_API_KEY from .env; visitors never see it.

Guardrail test: any advice at all means the guardrail failed. Paste the reply into the chat and tighten the system prompt.



A staff-only dashboard to run the clinic day

Role "reception" only

One seeded staff user; its password comes from RECEPTION_PASSWORD in .env.

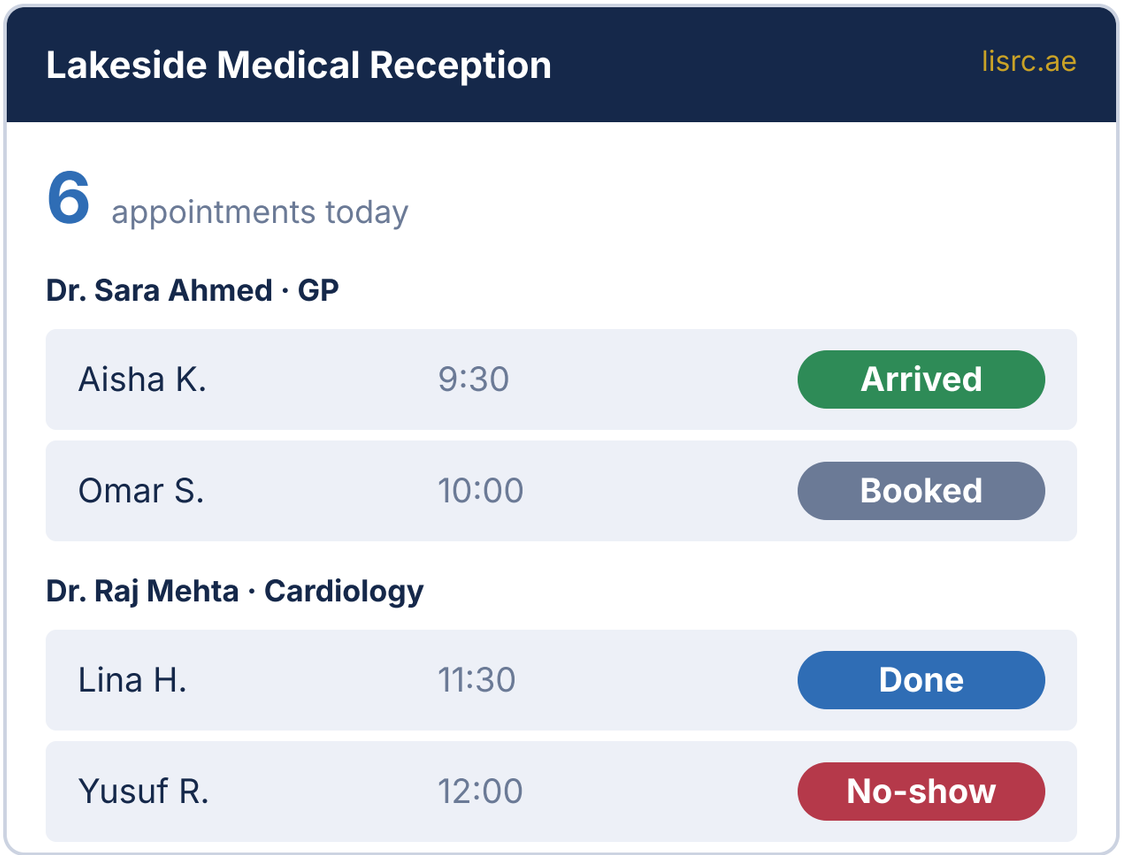
Today, grouped by doctor

Asia/Dubai time zone, with a count at the top.

One-tap status buttons

Arrived, Done, No-show write back through appointmentService.

Check: log in as a patient and confirm /reception is refused.



Polish for phones, then go live on Azure

P3 first: make the booking flow, My Appointments and Reception fully mobile-friendly, with a clear success message after booking.

1

Publish

Install the Azure App Service extension → Create Web App (Linux, Node 22 LTS, UAE North) → Deploy

2

Database

Set DB_PATH to /home/data/lakeside.db so bookings survive restarts

3

Secrets

Copy every .env line into the web app's Environment variables; set BASE_URL to the live URL

4

Domain

Connect a custom domain in App Service settings

5

Demo

Record 60–90s: book, pay, reception marks Arrived

P1 · Build the AI assistant (grounded + guardrail)

Add a floating chat assistant "**[assistant name — e.g. Ask Lakeside]**" (bottom-right, on every page, added in views/partials/footer.ejs). Build a server-side route POST /api/chat that sends the conversation to an AI provider's API (**[AI provider and model — e.g. the Claude API through the @anthropic-ai/sdk package, model claude-haiku-5-5]**). Read the API key from process.env.ANTHROPIC_API_KEY in .env — never in browser code. Put all AI code in one file called services/assistantService.js, and put the rules below in the system prompt. Answer using ONLY this info: Specialties — GP, ENT, Dentist, Cardiologist. Doctors — **[doctors — e.g. Dr. Sara Ahmed (GP), Dr. Omar Khan (ENT), Dr. Lina Haddad (Dentist), Dr. Raj Mehta (Cardiologist)]**. Hours — **[opening hours — e.g. Mon–Sat 9am–9pm, closed Sunday]**. Insurance — **[insurers — e.g. Daman, AXA, or cash]**. Location — **[address — e.g. Cluster X, JLT, Dubai]**. Deposit — **[deposit — e.g. AED 100]**. If the answer is not in this info, say you don't know and suggest calling the clinic. GUARDRAIL: do NOT give medical advice or discuss symptoms, diagnoses, or treatments. If asked, say you can't give medical advice, offer to book the right specialist, and suggest urgent/emergency care if it sounds serious. Keep answers short and friendly, and end each reply with a small one-line sign-off: "— **[clinic name — e.g. Lakeside Medical]**, powered by London International · lisrc.ae".

[Gold] = yours to fill in {Blue} = filled in automatically Everything else: keep as written

P2 · Build the reception dashboard

Upgrade the /reception page into a password-protected "Reception" dashboard. Add a role column to the users table in data/db.js (default "patient", only if it is missing). On startup, seed one staff user **[staff email — e.g. reception@lakeside.local]** with role "reception", with the password read from process.env.RECEPTION_PASSWORD in .env, stored hashed with bcryptjs. Add a middleware that allows only the "reception" role to open /reception. Show today's appointments (using the Asia/Dubai time zone) grouped by doctor — each row: patient name, time, specialty, status — with buttons to mark "Arrived", "Done", "No-show" (updating the database through appointmentService). Show a count of today's appointments at the top. Keep the "Send reminder" button. In the dashboard header add a small label "**[clinic name — e.g. Lakeside Medical]** Reception — London International · lisrc.ae", with "by **[your name — e.g. Aisha Khan]**" in the corner.

[Gold] = yours to fill in {Blue} = filled in automatically Everything else: keep as written

P3 · Mobile-friendly and success message

> Make the whole app fully mobile-friendly, including the booking flow, My Appointments and the Reception dashboard, and show a clear success message after booking.

[Gold] = yours to fill in {Blue} = filled in automatically Everything else: keep as written

YOUR SKILL TRANSFERS

Prompt, app, ship: on every platform

Lovable, Bolt, Base44, v0 and VS Code with Copilot or Claude Code all follow the same structure. Keep the 50+ reusable prompt library for every future build.

